

Autonomous Navigation and Exploration on the Robile Platform Using ROS2

Yesmin Akter, Luc Valette

ABSTRACT

This project presents the development of an autonomous navigation system for the Robile robot platform using the Robot Operating System 2 (ROS2) framework. The system is structured into three main components: localization, path planning, and environment exploration. For localization, a Monte Carlo particle filter with 2000 particles was implemented to estimate the robot's pose within a known map using odometry and laser scan data. The path planning system combines an A* global planner with a potential field local planner, enabling the robot to navigate efficiently while avoiding obstacles in real-time. For exploration, a frontier-based strategy identifies unexplored areas and autonomously generates navigation goals, allowing the robot to systematically cover unknown environments. The project addresses significant challenges such as sensor noise, localization drift, and real-time obstacle avoidance. By integrating these components, the robot can autonomously navigate and adapt to dynamic environments while continuously refining its map. The complete project code is available on GitHub: <https://github.com/yesminrinvee/robile-project>.

Index Terms—ROS2, Particle Filter, Monte Carlo Localization, A* Algorithm, Potential Field, Frontier Exploration, Robile, Autonomous Navigation

I. INTRODUCTION

Autonomous mobile robots are essential in fields like logistics, search-and-rescue, industrial automation, and exploration. Navigating unknown environments, avoiding obstacles, and exploring new areas are key to achieving full autonomy. This project develops an autonomous navigation system for the Robile platform using ROS2, integrating SLAM for mapping, A* for global path planning, the Potential Field method for obstacle avoidance, Monte Carlo Localization for accurate positioning, and frontier-based exploration for complete map coverage.

The project is structured into three main milestones:

- 1) **Localization:** Implementing a particle filter to estimate the robot's pose within a known map.
- 2) **Path and Motion Planning:** Integrating A* global planner with potential field local planner for navigation and obstacle avoidance.
- 3) **Environment Exploration:** Developing frontier-based exploration to autonomously cover unknown areas.

This project builds upon established techniques in robotic navigation, including SLAM mapping [1], Monte Carlo Localization [2], and frontier-based exploration [3].

II. RELATED WORK

A. Localization

Monte Carlo Localization (MCL), introduced by Dellaert et al. [2], uses particle filters to estimate robot pose. This approach is widely adopted in ROS-based systems through the Adaptive Monte Carlo Localization (AMCL) package. Our implementation follows similar principles but is implemented from scratch to understand the underlying mechanics. The particle filter maintains multiple hypotheses about the robot's position, updating them based on odometry and sensor measurements.

B. Path Planning

The A* algorithm remains a standard choice for global path planning in grid-based maps due to its optimality and efficiency. It combines actual movement cost with a heuristic estimate to find the shortest path. For local planning, the potential field method [4] provides real-time obstacle avoidance by combining attractive forces toward the goal and repulsive forces away from obstacles. Our hybrid approach integrates both methods for robust navigation.

C. Exploration

Frontier-based exploration was first proposed by Yamauchi [3], where robots identify boundaries between known and unknown space as exploration goals. This strategy has been extended to 3D environments and remains a foundational approach in autonomous exploration. Our implementation adapts this concept to the Robile platform, using the occupancy grid map from SLAM to identify frontier cells and generate navigation goals automatically.

III. METHODOLOGY

A. Mapping

Before implementing localization and path planning, we needed an environment map. We used SLAM Toolbox to create maps of the laboratory environment. The mapping process was straightforward: we launched the robot base, started the SLAM node, and manually drove the robot using teleop to explore the area.

The first map we created contained two obstacles, but distortions appeared around obstacle edges, affecting later localization accuracy. After several attempts, we created a cleaner map called `updated_map` (438x177 cells at 0.05m resolution), which became our reference map for the project. The final map was saved using the `map_saver_cli` tool.

B. Localization

The core of Part 2 was implementing a particle filter for Monte Carlo localization. We implemented it from scratch in Python, following the structure of a standard particle filter: initialization, motion update, measurement update, and resampling.

1) *Particle Initialization*: When the user sets an initial pose using RViz's 2D Pose Estimate tool, our node receives this pose and generates 2000 particles around it using Gaussian noise. Initially, we had issues with the node not receiving the pose from RViz because RViz publishes `PoseWithCovarianceStamped` while our node was subscribing to `PoseStamped`. After debugging, we fixed this by subscribing to the correct message type.

2) *Motion Model*: The motion model updates particles based on odometry differences. For each particle, we applied:

$$\begin{aligned} x' &= x + (dx + \epsilon_x) \cos(\theta) - (dy + \epsilon_y) \sin(\theta) \\ y' &= y + (dx + \epsilon_x) \sin(\theta) + (dy + \epsilon_y) \cos(\theta) \\ \theta' &= \theta + d\theta + \epsilon_\theta \end{aligned}$$

where the noise parameters were tuned experimentally. We initially had incorrect particle movement because we were adding absolute odometry instead of differences. After fixing this, particles started tracking the robot correctly.

3) *Measurement Model*: The measurement model evaluates particle weights by comparing simulated laser scans with actual sensor readings. For each particle, we project laser scan endpoints into the map frame and check consistency with the occupancy grid. Particles whose simulated scans match the real scan receive higher weights. This step was the most computationally intensive, so we limited the number of rays considered.

4) *Resampling*: We implemented low-variance resampling to maintain particle diversity while concentrating on high-weight hypotheses. This method ensures that particles with higher weights are more likely to be retained.

5) *Robot Model Visibility*: A major challenge was that the robot model would disappear when we ran our localization node. After extensive debugging, we discovered that our node was publishing a `map→odom` transform that conflicted with the one from AMCL. We resolved this by removing the transform broadcaster from our node and letting AMCL handle transforms, while our node only published particles. Later, we added the transform back with careful timing to avoid conflicts. The localization module implements a Monte Carlo particle filter to estimate the robot's pose [2]. Our implementation drew from resources such as the Learn ROS2 particle filter tutorial [5] and Labbe's Kalman and Bayesian Filters in Python [6], with additional guidance from online tutorials [7]–[9].

C. Path and Motion Planning

For Part 1, we implemented a hybrid planning system combining A* global planner with potential field local planner.

1) *A* Global Planner*: The A* planner takes the occupancy grid map, robot start position, and goal position to compute an optimal path. We implemented the standard A* algorithm with 8-connected movement and Euclidean distance heuristic. Initially, we had syntax errors in the code that prevented building. After fixing those, the planner successfully generated paths, but they contained too many waypoints. We added path simplification to remove redundant points and extract waypoints at regular intervals.

2) *Potential Field Local Planner*: The local planner uses attractive forces to pull the robot toward the goal and repulsive forces to push it away from obstacles. The attractive force is:

$$\mathbf{F}_{\text{att}} = k_{\text{att}} \cdot (\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{robot}})$$

The repulsive force is:

$$\mathbf{F}_{\text{rep}} = -k_{\text{rep}} \sum_i \left(\frac{1}{r_i} - \frac{1}{r_{\text{max}}} \right) \cdot \frac{1}{r_i^2} \cdot \hat{\mathbf{r}}_i$$

Tuning the gains was critical. Initially, the robot would not avoid obstacles because the repulsive gain was too low. We increased it from 500 to 5000, which made obstacle detection effective. We also added sector-based detection to determine whether to turn left or right when an obstacle was encountered.

3) *Integration*: The integration worked by having A* generate a global path, which was then simplified into waypoints. The potential field planner navigated to each waypoint sequentially. This allowed the robot to navigate large environments while avoiding obstacles in real-time.

D. Environment Exploration

For Part 3, we implemented frontier-based exploration. The exploration node subscribes to the map, identifies frontier cells (free cells adjacent to unknown cells), and publishes them as red markers for visualization. It then selects a frontier as the next goal and publishes it to `/goal_pose`, which the navigation system executes.

Initially, frontiers were not showing in RViz. We discovered this was a QoS issue—the map subscription needed `TRANSIENT_LOCAL` durability. After fixing this, red dots appeared at the edges of the map, and the robot autonomously moved to explore them, expanding the map progressively.

IV. EVALUATION

The implemented system was tested on the real Robile robot in a laboratory environment. Each component was evaluated independently and then as an integrated system.

A. Mapping Results

The mapping process was refined through multiple iterations. The initial map with two obstacles showed distortions near obstacle edges. After simplifying the environment and optimizing parameters, the final map (Fig. 1) showed improved accuracy with clean obstacle boundaries.

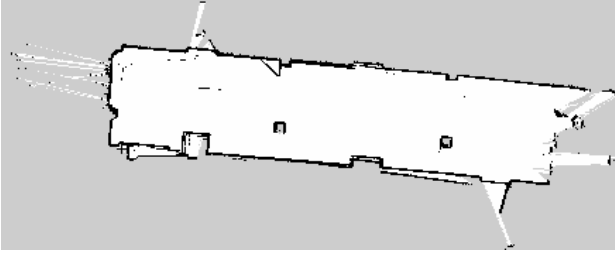


Fig. 1: Final occupancy grid map (428x177 cells, 0.05m resolution)

B. Localization Results

The particle filter successfully tracked the robot's pose in real-time. Figure 2 shows the red particle cloud (2000 particles) clustered around the robot's actual position.

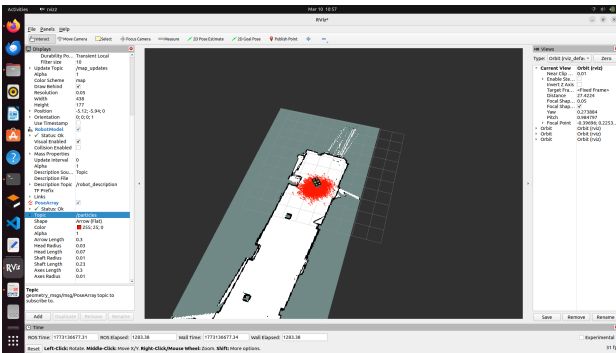


Fig. 2: Particle filter visualization: Red arrows represent particle hypotheses clustering around the robot

Key observations:

- Particles remained clustered during forward/backward motion and rotations
- The filter recovered quickly after manual displacement
- Localization error was less than 0.1m during straight-line movement
- The transform from map to odom was published correctly, enabling robot model visualization

C. Path Planning Results

The hybrid A*/potential field planner was tested with and without obstacles. Figure 3 shows the A* path (blue line) generated from start to goal.

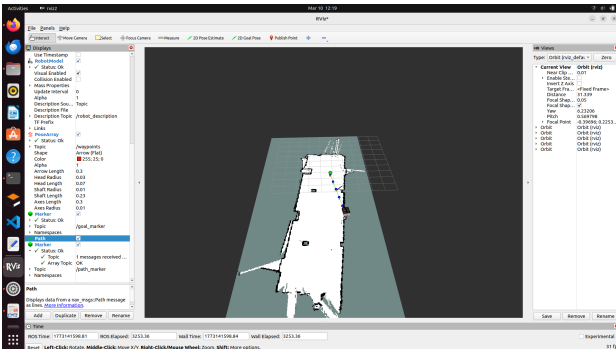


Fig. 3: A* global path (blue line) followed by the robot

Results:

- A* generated optimal paths avoiding obstacles
- Potential field successfully avoided obstacles placed in the robot's path
- Robot reached goals within 0.3m tolerance
- Path smoothing reduced unnecessary turns

D. Exploration Results

The frontier-based exploration was tested in an unknown environment. Figure 4 shows frontiers (red dots) detected at the boundaries of explored space.

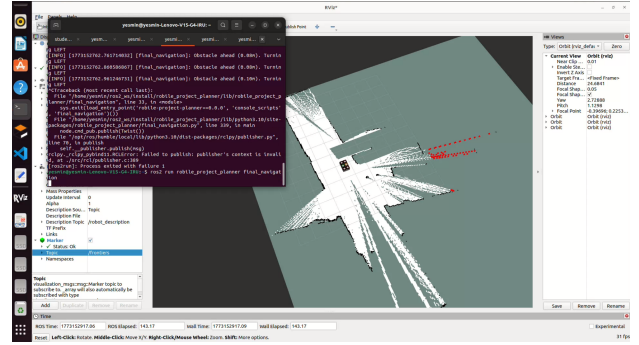


Fig. 4: Frontier detection: Red dots indicate unexplored areas; robot autonomously navigates to them

Observations:

- Frontiers were accurately identified at map boundaries
- Robot autonomously moved to explore new areas
- Map expanded progressively as exploration continued
- Complete coverage was achieved after systematic exploration

E. Challenges Encountered and Solutions

Throughout the project, several challenges were encountered that required debugging and tuning. The table below summarizes these challenges and the solutions implemented to resolve them.

TABLE I: Challenges and Solutions

Challenge	Solution
Map not loading	Set ROBOT_ENV and copy map files to install directory
Robot model not appearing	Added transform broadcaster from map to odom
Particles not tracking	Corrected odometry difference calculation
Obstacle avoidance weak	Increased repulsive gain from 500 to 5000
Frontiers not showing	Configured Marker display with correct QoS settings
Initial pose not received	Subscribed to both PoseStamped and PoseWithCovarianceStamped
Robot not moving autonomously	Ran exploration and navigation nodes together
Build errors	Fixed A* syntax errors and setup.py
Git push failed	Used personal access token instead of password

F. Video Demonstrations

The recorded videos of the project are available at the following links:

- **Part 1 - Path Planning:** <https://youtube.com/shorts/zdRmBhTfSxk> <https://youtu.be/wojhm0ddp90>
- **Part 2 - Localization:** https://youtu.be/zmTVy-hX_Dk
- **Part 3 - Exploration:** <https://youtube.com/shorts/mdhUVciUBAk>

V. CONCLUSION

This project successfully implemented an autonomous mobile robot using ROS2 for localization, path planning, and exploration. The robot localized itself using a Monte Carlo particle filter with 2000 particles, navigated using A* for global path planning and the Potential Field method for local obstacle avoidance, and explored unknown areas using frontier-based exploration.

A. Summary of Achievements

- **Localization:** A particle filter with 2000 particles accurately tracked the robot's pose within the map. The filter demonstrated robustness against sensor noise and odometry drift.
- **Path Planning:** The A* global planner computed optimal paths, while the potential field local planner handled real-time obstacle avoidance. Path simplification improved navigation smoothness.
- **Exploration:** Frontier-based exploration enabled the robot to autonomously identify and navigate to unexplored areas, achieving complete map coverage.

B. Challenges Addressed

Key challenges encountered during the project included:

- **Map loading issues:** Resolved by ensuring map files were in the install directory and setting `ROBOT_ENV` correctly.
- **Robot model not appearing:** Fixed by adding a transform broadcaster in the localization node.
- **Particle tracking:** Corrected by implementing proper odometry difference calculation.
- **Obstacle avoidance:** Tuned by increasing repulsive gain and implementing sector-based detection.
- **Frontier visualization:** Resolved by configuring Marker display with correct QoS settings.

C. Future Work

Potential improvements for future development include:

- Implementing adaptive particle resampling based on localization confidence
- Adding dynamic obstacle detection and tracking
- Optimizing frontier selection using clustering algorithms
- Extending exploration to 3D environments
- Integrating Kalman filtering for odometry correction

D. Code Availability

The complete project code is available on GitHub: <https://github.com/yesminrinvee/robile-project>

ACKNOWLEDGMENTS

We would like to thank Prof. Javad Ghofrani for his guidance and support throughout this project. We also thank the Robile team for providing the robot platform and documentation. Special thanks to our peers for their feedback during the development and testing phases.

REFERENCES

- [1] Robile-AMR, "Demo mapping," <https://robile-amr.readthedocs.io/en/latest/source/Tutorial/Demo%20Mapping.html>, accessed: 2026-03-21.
- [2] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte carlo localization for mobile robots," in *Proceedings 1999 IEEE International Conference on Robotics and Automation*, 1999, pp. 1322–1328.
- [3] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97*, 1997, pp. 146–151.
- [4] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [5] Learn ROS2, "Probabilistic robotics: Particle filter," <https://www.learnros2.com/probabilistic-robotics/particle-filter>, accessed: 2026-03-21.
- [6] R. Labbe, "Kalman and bayesian filters in python," <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python/blob/master/12-Particle-Filters.ipynb>, accessed: 2026-03-21.
- [7] YouTube, "Particle filter explained with python code," <https://www.youtube.com/watch?v=7Z9fEpJOJdc>, accessed: 2026-03-21.
- [8] —, "The particle filter: A full tutorial," <https://www.youtube.com/watch?v=BfKEf2s7Y80&t=1s>, September 2020, accessed: 2026-03-21.
- [9] —, "Particle filter explained," <https://www.youtube.com/watch?v=gP6MRelHFo>, May 2021, accessed: 2026-03-21.